

Of course! Here are the detailed answers to your questions, based on the provided documents.

1. Explain the concept of Object-Oriented Programming with its basic principles.

Object-Oriented Programming (OOP) is a very important programming style that is based on the concept of "objects"¹. Instead of just writing procedures or functions, OOP allows us to bundle data and the methods that work on that data into a single unit called an object. This makes our code more organized, reusable, and easier to manage.

The basic principles of OOP are:

♦ Abstraction

Abstraction is the process of hiding the complex implementation details from the user and only showing the essential features or functionality².

- **Analogy** 🤔: Think about a TV remote. You just press the "volume up" button to increase the volume. You don't need to know how the remote sends the signal or how the TV processes it. The complex part is hidden, and you are only shown the simple button. In Java, we use **abstract classes** and **interfaces** to achieve abstraction³.

♦ Encapsulation

Encapsulation is the technique of wrapping up data (variables) and the code that acts on the data (methods) together as a single unit, like a capsule⁴. It also involves controlling access to the data using access modifiers like

private, which prevents outside code from accidentally changing the data.

- **Analogy** 📁: A school bag is a good example. You keep your books, pens, and lunchbox (data) inside the bag (class). The bag protects them from the outside world.

♦ Inheritance

Inheritance is a powerful feature that allows a new class (called a

subclass or **child class**) to acquire the properties and methods of an existing class (called a **superclass** or **parent class**)⁵⁵⁵. This is super useful for code reusability⁶⁶⁶⁶.

- **Analogy** 👨👩: A child inherits traits like eye color and hair color from their parents. Similarly, in Java, a Dog class can inherit properties like color and methods like eat() from an Animal class.

♦ Polymorphism

The word polymorphism means "many forms." In programming, it is the ability of a single task to be performed in different ways⁷. This allows us to perform a single action in different ways.

- **Example** 🐶🐱: Let's say we have an Animal class with a sound() method. A Dog object and a Cat object can both have a sound() method, but the Dog will "bark" and the Cat will "meow." This is achieved through **method overriding**⁸.

2. Explain the history and features of Java.

The provided documents do not contain information about the history of Java. However, they highlight several key features of the language through various concepts.

The main features of Java are:

- ♦ **Object-Oriented**

Java is fundamentally an Object-Oriented Programming (OOP) language⁹. Everything in Java is associated with classes and objects. It supports all the core OOP principles like Inheritance, Encapsulation, Abstraction, and Polymorphism, which help in building robust and modular applications¹⁰¹⁰¹⁰¹⁰¹⁰¹⁰¹⁰.

- ♦ **Platform Independent**

This is one of Java's most famous features! Java programs are compiled into an intermediate format called bytecode. This bytecode is not specific to any processor and can be run on any machine that has a

Java Virtual Machine (JVM) installed¹¹. The JVM acts as a software engine that executes the Java bytecode, making Java programs "write once, run anywhere"¹².

- ♦ **Multithreading**

Java has built-in support for multithreading, which allows a program to perform multiple tasks concurrently¹³¹³. A thread is the smallest unit of processing¹⁴. Multithreading is very useful for creating responsive applications and improving performance, especially in games or applications with a graphical user interface (GUI).

3. Describe the structure of a Java program with an example.

A typical Java program has a well-defined structure that helps in organizing the code. The main parts are:

1. **Package Statement (Optional):** The `package` keyword is used to declare a package¹⁵. A package is a way to group related classes and interfaces¹⁶¹⁶. This must be the very first statement in the file.
2. **Import Statements (Optional):** The `import` keyword is used to access classes and interfaces from another package into the current file¹⁷.
3. **Class Declaration:** Every Java program must have at least one class definition. A class is a blueprint for creating objects. It starts with the `class` keyword.
4. **The main Method:** This is the entry point of any Java application. The JVM starts executing the program from the main method. It has a specific signature:
`public static void main(String[] args)`¹⁸¹⁸¹⁸¹⁸.

Example Program Structure

Here is an example that demonstrates the structure of a Java program, combining elements from the provided documents.

Java

```
// 1. Package Statement
package mypack; [cite: 1196]

// 2. Import Statement
import pack.*; [cite: 1232]

// 3. Class Declaration
public class Simple { [cite: 1197]

// 4. The main() Method
    public static void main(String args[]) { [cite: 1198]
        // Creating an object of class A from the 'pack' package
        A obj = new A(); [cite: 1239]
```

```
obj.msg(); // Calling a method on the object [cite: 1243]

System.out.println("Welcome to my Java program!"); [cite: 1199]
}
}
```

Explanation:

- package mypack; declares that this class Simple belongs to the package named mypack.
- import pack.*; makes all the public classes from the pack package available to use in this file.
- public class Simple { ... } defines a public class named Simple.
- public static void main(String args[]) { ... } is the main method where the program execution begins.

4. Explain different data types in Java with examples.

A data type specifies the different sizes and values that can be stored in a variable. The provided documents primarily use two types of data: int for integers and String for text.

1. int (Integer Type):

- **Definition:** This is a primitive data type used to store whole numbers (numbers without a decimal point), such as 10, -5, or 100.
- **Example:** In the code below, the variable result is declared as an int to hold the result of a calculation.

```
Java
int result = square(5); // 'result' is an integer variable [cite: 14]
```

- Another example shows an array of integers being created:

```
Java
int a[] = new int[5]; // An array that can hold 5 integers [cite: 415]
```

2. String (Non-Primitive Type):

- **Definition:** String is a class in Java used to store a sequence of characters, like a name or a message. It is not a primitive type but an object.
- **Example:** Here, a String variable name is declared.

```
Java
public String name; // 'name' is a String variable [cite: 234]
```

- Strings are also used for displaying text messages:

```
Java
```

```
System.out.println("Square = " + result); // "Square = " is a String literal [cite: 15]
```

Type Casting

Sometimes, you need to convert a value from one data type to another. This is called

type casting¹⁹.

- **Example:** To convert a decimal number 10.5 into an integer, you can cast it like this:

Java

```
int x = (int) 10.5; // The value of x will be 10
```

5. Write short notes on Command Line Arguments with example.

The provided documents show the standard signature for the main method in Java, which is the entry point for program execution:

```
public static void main(String[] args)20202020.
```

The parameter `String[] args` in this method is designed to receive **command-line arguments**. This means you can pass information directly to your Java program when you run it from the command line.

- `String[]`: This indicates that the arguments are received as an array of String objects.
- `args`: This is the name of the array. You can use any name, but `args` is the conventional choice.

While the syntax `(String[] args)` is present in many examples across the documents, the documents themselves **do not provide an explanation or a code example of how to access and use these command-line arguments** within the program. The `args` array is

never used in the provided code snippets.

6. Explain switch Statements.

The

switch statement is a control flow statement used in Java to select one of several code blocks to be executed based on the value of a variable or an expression²¹. It provides a cleaner alternative to a long series of

if-else if-else statements, especially when you need to compare a single variable against multiple possible values.

Key Components of a switch Statement:

- **switch (expression):** The expression is the variable or value that is being tested. It must evaluate to a byte, short, char, int, String, or an enum.
- **case value::** Each case label represents one possible value for the expression. If the expression's value matches a case value, the code block following that case is executed.
- **break::** The break keyword is used to exit the switch block. If you forget to add a break, the execution will "fall through" to the next case block, which is usually not what you want.
- **default::** The default block is optional. It is executed if none of the case values match the expression's value.

While the purpose of the

switch statement is defined²²,

no code examples of a switch statement are available in the provided documents to demonstrate its usage.

7. Explain class, object, and static members with

examples.

In Java, classes, objects, and static members are fundamental concepts of object-oriented programming.

♦ Class

A

class is a blueprint or a template from which objects are created²³²³. It defines a set of properties (variables) and behaviors (methods) that are common to all objects of that type.

- **Example:** Here is a simple class named Nesting that defines two methods.

```
Java
class Nesting { // This is a class definition [cite: 7]
    int square(int n) { [cite: 8]
        return n * n; [cite: 9]
    }

    void display() { [cite: 13]
        int result = square(5); [cite: 14]
        System.out.println("Square = " + result); [cite: 15]
    }
}
```

♦ Object

An

object is an instance of a class²⁴. When a class is defined, no memory is allocated; memory is only allocated when an object of that class is created using the

new keyword. You can create multiple objects from the same class blueprint.

- **Example:** Here, an object named obj is created from the Nesting class.

```
Java
// 'obj' is an object (or instance) of the Nesting class
Nesting obj = new Nesting(); // object created [cite: 19]

// We can now use the object to call the methods defined in the class
obj.display(); // method called by object [cite: 20]
```

◆ Static Members

A **static** member (variable or method) belongs to the class itself rather than to any individual object. This means you can access a static member without creating an object of the class.

- **Static Method Example:** The main method in Java is always static. This allows the JVM to call it to start the program without needing to create an object first.

```
Java
public static void main(String[] args) { // 'main' is a static method [cite: 17]
    // ...
}
```

- **Static Variable Example:** In an interface, constants are implicitly public, static, and final²⁵²⁵²⁵²⁵. This means the value is shared across all implementations and can be accessed using the interface name.

```
Java
interface Vehicle {
    // This is a public static final constant by default [cite: 184]
    int MAX_SPEED = 120;
}
```

8. Describe constructors in Java with types and examples.

A **constructor** in Java is a special type of method that is used to initialize an object when it is created. It has the same name as the class and does not have a return type.

The main types of constructors are:

◆ Default Constructor

A default constructor is a constructor that has no parameters²⁶. If you do not define any constructor in your class, the Java compiler automatically provides a default constructor for you²⁷.

- **Example:** In the Nesting class example, no constructor was defined. So, when we write `new Nesting()`, we are using the default constructor provided by the compiler.

Java

```
Nesting obj = new Nesting(); // Calls the default constructor [cite: 19]
```

◆ Parameterized Constructor

A parameterized constructor is a constructor that accepts one or more parameters. These are used to initialize the object with specific values at the time of creation.

- **Example:** The Thread class in Java has several constructors, including parameterized ones. The example below uses a constructor that takes a Runnable object (`this`) and a String name to create a new thread.

Java

```
// This is an example of calling a parameterized constructor  
thread = new Thread(this, name); [cite: 1773]
```

Another example is the Vector class, which has a constructor to set its initial capacity:

```
java // vector(int initialCapacity) is a parameterized constructor [cite: 1624]  
Vector myVector = new Vector(20); // Creates a vector with an initial size of 20
```

It's important to note that

interfaces cannot have constructors because they cannot be instantiated directly²⁸²⁸.

9. Explain single and multilevel inheritance with example.

Inheritance is a mechanism where one class acquires all the properties and behaviors of a parent class²⁹²⁹²⁹. Java supports several types of inheritance, including single and multilevel.

◆ Single Inheritance

In single inheritance, a class inherits from only one parent class. This is the simplest form of inheritance.

- **Explanation:** There is one superclass (parent) and one subclass (child). The child class extends the functionality of the parent class.
- **Example:** Let's consider an Animal superclass and a Dog subclass. The Dog class inherits the eat() method from Animal and also has its own bark() method.

Java

```
// Parent class (Superclass)
class Animal { [cite: 35]
    void eat() { [cite: 36]
        System.out.println("Eating..."); [cite: 37]
    }
}

// Child class (Subclass) inherits from Animal
class Dog extends Animal { [cite: 42]
    void bark() { [cite: 43]
        System.out.println("Barking..."); [cite: 44]
    }
}
```

```

}

public class Main {
    public static void main(String[] args) { [cite: 48]
        Dog d = new Dog(); [cite: 49]
        d.eat(); // Calling method from parent class [cite: 50]
        d.bark(); // Calling method from its own class [cite: 51]
    }
}

```

◆ Multilevel Inheritance

In multilevel inheritance, a class inherits from another class, which in turn inherits from another class, forming a chain.

- **Explanation:** It involves a chain of inheritance, like a grandchild inheriting from a parent, who inherits from a grandparent. (e.g., Class C -> Class B -> Class A).
- **Example:** The hierarchy of the Applet class in Java is a perfect example of multilevel inheritance.

The hierarchy is as follows:

JApplet -> Applet -> Panel -> Container -> Component -> Object

Here's how it works:

- The Applet class extends the Panel class.
- The Panel class extends the Container class.
- The Container class is a subclass of the Component class³⁰.

This creates a chain of inheritance, where Applet inherits properties and methods from Panel, Container, and Component up the chain.

10. Explain the concept of interfaces with example.

An

interface in Java is a blueprint of a class³¹. It acts as a contract that a class can agree to

implement. It is a way to achieve

total abstraction³² and is also used to support the functionality of

multiple inheritance in Java³³.

Key Points about Interfaces:

- **Abstract Methods:** An interface can only have abstract methods, which are methods without a body³⁴³⁴³⁴³⁴. A class that implements the interface must provide the implementation for all its methods³⁵.
- **No Constructors:** Interfaces cannot have constructors and cannot be instantiated directly³⁶³⁶³⁶³⁶.
- **implements Keyword:** A class uses the implements keyword to use an interface. A class can implement multiple interfaces³⁷.
- **Constants:** Variables declared in an interface are public, static, and final by default, meaning they are constants³⁸³⁸³⁸³⁸.
- **Default Methods (from Java 8):** Interfaces can have default methods that include a body. A class can use this method directly without needing to implement it³⁹.

Example

Here is an example of an Animal interface that is implemented by Dog and Cat classes.

Java

```
// 1. Define an interface
interface Animal { [cite: 125]
    // Abstract method (no body)
    public void sound(); [cite: 127]

    // Default method (has a body)
    default void sleep() { [cite: 129]
        System.out.println("This animal is sleeping."); [cite: 130]
    }
}

// 2. Implement the interface in a class
class Dog implements Animal {
    // Providing the body for the abstract sound() method
    public void sound() { [cite: 137]
        System.out.println("Woof"); [cite: 138]
    }
}

// 3. Implement the interface in another class
class Cat implements Animal { [cite: 141]
    // Providing the body for the abstract sound() method
    public void sound() { [cite: 143]
        System.out.println("Meow"); [cite: 146]
    }
}

// 4. Using the classes
public class Main {
    public static void main(String[] args) { [cite: 149]
        Dog myDog = new Dog(); [cite: 151]
        myDog.sound(); // Output: Woof [cite: 154, 155]
        myDog.sleep(); // Output: This animal is sleeping. [cite: 156, 157]

        Cat myCat = new Cat(); [cite: 152]
        myCat.sound(); // Output: Meow [cite: 158, 159]
        myCat.sleep(); // Output: This animal is sleeping. [cite: 160, 161]
    }
}
```

11. Write short notes on Java Packages.

A

Java package is a way of grouping a variety of similar classes, interfaces, and sub-packages into a single container⁴⁰⁴⁰⁴⁰⁴⁰. Think of it like a folder on your computer that you use to keep related files organized. Packages can be built-in (provided by Java) or user-defined (created by the programmer)⁴¹.

Advantages of Using Packages:

1. **Organization:** Packages help in categorizing classes and interfaces so they can be easily maintained and located⁴².
2. **Access Protection:** Packages provide access protection. You can define classes and members that are only visible within the package (using default or protected access)⁴³.
3. **Removes Naming Collisions:** Packages help prevent naming conflicts⁴⁴. For example, you can have a class named

List in two different packages (java.awt.List and java.util.List).

Creating and Using a Package:

1. Creating a Package:

To create a package, you use the package keyword as the first statement in your Java file⁴⁵.

Java

```
// File saved as Simple.java
package mypack; // This class is now part of the 'mypack' package [cite: 1196]

public class Simple { [cite: 1197]
    public void msg() {
        System.out.println("Hello from mypack!");
    }
}
```

2. Using a Package:

There are three ways to access a class from another package⁴⁶:

- `import package.*;` This imports all public classes and interfaces from the specified package⁴⁷.
- `import package.classname;` This imports only the specified class⁴⁸.
- **Fully qualified name:** This involves using the full package name every time you use the class (e.g., `pack.A obj = new pack.A();`).

Example of importing a package:

Java

```
// File saved as B.java
package anotherpackage;

import mypack.*; // Imports all classes from mypack [cite: 1232]

class B {
    public static void main(String args[]) { [cite: 1237]
        Simple obj = new Simple(); // Now we can use the Simple class
        obj.msg();
    }
}
```

12. Explain the lifecycle of a thread with suitable

diagram.

A

thread is a lightweight sub-process that allows for concurrent execution within a single program⁴⁹⁴⁹⁴⁹. Just like a process, a thread goes through several stages from its creation to its termination. This sequence of stages is known as the

lifecycle of a thread.

The states in a thread's lifecycle are⁵⁰⁵⁰⁵⁰⁵⁰:

1. New:

- A thread is in the **New** state when an instance of the Thread class has been created but the start() method has not yet been called⁵¹⁵¹⁵¹⁵¹. At this point, the thread is not yet alive.

2. Runnable (Ready to Run):

- After the start() method is invoked on a new thread, it enters the **Runnable** state⁵²⁵²⁵²⁵². In this state, the thread is ready to run and is waiting for the thread scheduler to allocate CPU time⁵³. A thread in this state might be running, or it might be ready to run at any moment⁵⁴.

3. Running:

- A thread is in the **Running** state when the thread scheduler has selected it for execution and it is actively using the CPU. A thread moves from the Runnable state to the Running state⁵⁵.

4. Blocked / Waiting:

- A thread enters the **Blocked** or **Waiting** state when it is temporarily inactive⁵⁶⁵⁶⁵⁶⁵⁶. This can happen for several reasons, such as waiting for an I/O operation to complete or waiting to acquire a lock on an object. The thread moves back to the Runnable state once the condition it was waiting for is met.

5. Timed Waiting:

- This state is similar to the Waiting state, but the thread will wait only for a specified period of time⁵⁷. A thread enters this state when it calls a method with a timeout,

such as

`Thread.sleep()` or `wait(timeout)`. It returns to the `Runnable` state when the timeout expires or a notification is received⁵⁸.

6. **Terminated (Dead):**

- A thread enters the **Terminated** state when it has finished its execution, meaning its `run()` method has completed⁵⁹. This can happen either because the task finished normally or due to an unhandled exception⁶⁰. Once a thread is terminated, it cannot be started again.

13. What are the main features of Java? Explain any three.

The main features of Java that make it a powerful and popular programming language include being object-oriented, platform-independent, and multithreaded.

1. ☕ **Object-Oriented**

Java is a true Object-Oriented Programming (OOP) language⁶¹. This means it is organized around the concept of "objects," which bundle data and methods together. It strongly supports the four major principles of OOP:

- **Inheritance:** Allows a class to reuse the code from another class⁶².
- **Encapsulation:** Wraps data and methods into a single unit (class) to hide data from the outside world⁶³.
- **Abstraction:** Hides complex implementation details and shows only the necessary features⁶⁴.
- **Polymorphism:** Allows an object to take on many forms, for example, by letting a child class redefine a method from its parent class (method overriding)⁶⁵.

2. 🖥️ Platform Independent

Java's platform independence is one of its most significant advantages. When a Java program is compiled, it is converted into platform-neutral **bytecode**. This bytecode can be executed on any platform (like Windows, macOS, or Linux) that has a

Java Virtual Machine (JVM)⁶⁶. The JVM interprets the bytecode and translates it into machine code for the specific operating system. This principle is often summarized as "Write Once, Run Anywhere" (WORA).

3. 🏃 Multithreading

Java has built-in support for

multithreading, which is the ability of a program to execute multiple threads simultaneously⁶⁷⁶⁷⁶⁷. A thread is like a mini-process within a larger process⁶⁸. This feature is crucial for developing high-performance and responsive applications, such as animations, games, or complex server-side applications, as it allows multiple operations to proceed at the same time without blocking each other.

14. Explain variables and data types in Java with example.

In Java, **variables** are used to store data, while **data types** specify the type of data a variable can hold.

Variables

A variable is a container that holds a value. It's essentially a named piece of memory. Before you can use a variable in Java, you must declare it by specifying its data type and giving it a name.

- **Declaration:** type variableName;
- **Initialization:** variableName = value;
- **Combined:** type variableName = value;

Data Types

A data type defines the kind of values a variable can store and the operations that can be performed on it. The provided documents show examples of the following data types:

♦ int

- **Description:** A primitive data type used for storing 32-bit signed integer (whole number) values.
- **Example:** Here, age is a variable of type int.

Java

```
private int age; // Declaring an integer variable named 'age' [cite: 225]
```

```
int data = 100/0; // 'data' is an int variable initialized with a calculation result [cite: 389]
```

♦ String

- **Description:** A non-primitive data type (a class) used for storing sequences of characters (text).
- **Example:** Here, name is a variable of type String.

Java

```
public String name; // Declaring a String variable named 'name' [cite: 234]
```

```
String s = "abc"; // Declaring and initializing a String variable 's' [cite: 410]
```

Example Combining Variables and Data Types

This code snippet shows how variables of different data types are declared and used within a method.

Java

```
class Student {  
    public String name; // String variable to store a name [cite: 234]  
    private int age;    // int variable to store an age [cite: 225]  
  
    public void showDetails() {  
        // Using the variables  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

15. Write a short note on control statements in Java.

Control statements in Java are keywords that allow you to control the flow of execution of your program. They let you make decisions, repeat actions, and jump from one part of the code to another. They are broadly categorized into three types:

1. Decision-Making (Selection) Statements

These statements allow the program to choose a path of execution based on a condition.

- **if-else statement:** Executes a block of code if a condition is true, and another block if it's false. The else part is optional.

- **Example:**

```
Java
if (age < 18) { [cite: 441]
    // This code runs if age is less than 18
    throw new ArithmeticException("Person is not eligible to vote"); [cite: 443]
} else { [cite: 445]
    // This code runs otherwise
    System.out.println("Person is eligible to vote!!"); [cite: 446]
}
```

- **switch statement:** Allows a variable to be tested for equality against a list of values. It's useful for selecting one of many code blocks to be executed⁶⁹.

2. Looping (Iteration) Statements

These statements are used to execute a block of code repeatedly as long as a certain condition is true.

- **for loop:** Executes a block of code a specific number of times. It's great when you know the number of iterations in advance.

- **Example:**

```
Java
// This loop runs 5 times, with i going from 1 to 5
for(int i=1; i<=5; i++) { [cite: 96]
    System.out.println("Iteration: " + i);
}
```

- **while loop:** Executes a block of code as long as a given condition remains true.
- **do-while loop:** Similar to a while loop, but it executes the code block at least once before checking the condition.

3. Jump Statements

These statements transfer control to another part of the program.

- **break:** Terminates the loop or switch statement.
 - **continue:** Skips the current iteration of a loop and proceeds to the next one.
 - **return:** Exits from the current method.
-

16. Explain arrays in Java with example.

An **array** is a simple data structure that stores a fixed-size collection of elements of the **same data type**. Think of it as a row of containers, where each container holds a value, and you can access each container by its position (index).

Key Features of Arrays:

- **Fixed Size:** Once an array is created, its size cannot be changed.
- **Homogeneous Elements:** An array can only store elements of the same data type (e.g., all int or all String).
- **Index-Based:** Elements are stored in contiguous memory locations and are accessed using an index. The index starts from **0** and goes up to **size - 1**.

Declaring and Creating an Array

You declare an array by specifying the data type followed by square brackets []. You create it using the new keyword and specifying its size.

- **Syntax:** dataType[] arrayName = new dataType[size];
- **Example:** This creates an array named a that can hold 5 integer elements. The valid indices are 0, 1, 2, 3, and 4.

Java

```
int a[] = new int[5]; [cite: 415]
```

Accessing Array Elements

You access an element by using the array name followed by the index in square brackets.

- **Example:**

Java

```
a[0] = 10; // Assigns the value 10 to the first element  
int firstElement = a[0]; // Reads the value of the first element
```

ArrayIndexOutOfBoundsException

Because arrays have a fixed size, you will get a runtime error called

ArrayIndexOutOfBoundsException if you try to access an element with an index that is outside the valid range (i.e., less than 0 or greater than or equal to the array's size)⁷⁰.

- **Example:** The following code will cause this exception because the array a has indices from 0 to 4, but the code tries to access index 10.

Java

```
int a[] = new int[5]; [cite: 415]  
a[10] = 50; // This will throw an ArrayIndexOutOfBoundsException [cite: 416]
```

17. Discuss break and continue statements with example.

The break and continue statements are jump statements used to alter the normal flow of control within loops and switch statements.

break Statement

- **Purpose:** The break statement is used to terminate the execution of the innermost loop (for, while, do-while) or a switch statement immediately. When break is encountered, the

control of the program jumps to the statement immediately following the loop or switch block.

- **Common Uses:**

1. To exit a switch case, preventing "fall-through." The switch statement is defined as a way to select one block of code from multiple options⁷¹, and

break is essential for this functionality.

2. To exit a loop prematurely when a certain condition is met.

continue Statement

- **Purpose:** The continue statement is used inside loops. When it is encountered, it skips the rest of the statements in the current iteration of the loop and immediately jumps to the beginning of the next iteration.
- **Common Uses:**
 - To skip processing for a specific value within a loop without terminating the entire loop.

Note: The provided PDF documents **do not contain any code examples or detailed explanations** for the break and continue statements. Their functionality is described here based on standard Java principles, but no specific examples can be cited from the source material.

18. Explain nested if-else with suitable example.

A **nested if-else** statement is when an if or else statement contains another if or if-else statement inside its block. This allows for more complex, multi-layered decision-making in a program. You can check for a condition, and if it's true, you can then check for another condition.

Structure of Nested if-else

Java

```
if (condition1) {  
    // Code to execute if condition1 is true  
  
    if (condition2) {  
        // Code to execute if both condition1 and condition2 are true  
    } else {  
        // Code to execute if condition1 is true but condition2 is false  
    }  
} else {  
    // Code to execute if condition1 is false  
}
```

Explanation

The program first evaluates condition1.

- If condition1 is **true**, it enters the outer if block and then evaluates condition2. The inner if-else then determines the next step.
- If condition1 is **false**, it skips the entire outer if block and executes the outer else block.

The provided documents contain a simple

if-else example to check a person's eligibility to vote⁷²⁷²⁷²⁷², but they

do not include a specific example of a nested if-else statement.

To illustrate the concept, we can extend that example:

Java

```
// This is an illustrative example, not from the provided PDFs.  
int age = 20;
```

```
boolean hasVoterId = false;

if (age >= 18) {
    System.out.println("You are old enough to vote.");

    if (hasVoterId) {
        System.out.println("You can vote!");
    } else {
        System.out.println("But you need to get a Voter ID card first.");
    }
} else {
    System.out.println("You are not old enough to vote.");
}
```

In this illustrative logic, the program first checks the age. If the person is 18 or older, it then makes a nested check to see if they have a Voter ID.

19. Explain method overriding with example.

Method overriding is a feature of object-oriented programming that allows a subclass (child class) to provide a specific implementation for a method that is already defined in its superclass (parent class)⁷³. This is a key way to achieve

runtime polymorphism in Java⁷⁴.

Rules for Method Overriding:

For a method to be successfully overridden, it must follow these rules:

1. The method in the child class must have the **same name** as the method in the parent class.
2. The method must have the **same parameters** (number of parameters and their types) as in the parent class.

3. The method must have the **same return type** (or a subtype of the parent's return type).

The `@Override` annotation is often used above the overridden method. While not mandatory, it helps the compiler verify that you are actually overriding a method correctly.

Example

Here is a classic example where a `Dog` class overrides the `sound()` method of its parent `Animal` class.

Java

```
// 1. Parent Class
class Animal { [cite: 957]
    void sound() { [cite: 958]
        // Generic implementation
        System.out.println("Animal makes sound"); [cite: 951]
    }
}

// 2. Child Class
class Dog extends Animal { [cite: 959]
    // Overriding the sound() method of the Animal class
    @Override [cite: 960]
    void sound() { [cite: 961]
        // Specific implementation for Dog
        System.out.println("Dog barks"); [cite: 962]
    }
}

// 3. Test Class
public class Test {
    public static void main(String[] args) { [cite: 967]
        // Parent reference, but holding a child object
        Animal a = new Dog(); [cite: 968]

        // The JVM decides at RUNTIME which sound() method to call.
```

```
// Since 'a' refers to a Dog object, the Dog's sound() method is called.  
a.sound(); // Output: Dog barks [cite: 969]  
}  
}
```

In this example, even though the reference variable `a` is of type `Animal`, the object it holds is of type `Dog`. Due to runtime polymorphism, the overridden method in the `Dog` class is executed.

20. What are access modifiers? Explain with example.

Access modifiers in Java are keywords that set the accessibility or scope (visibility) of a class, constructor, variable, or method. They are a core part of encapsulation. Java has four access modifiers:

1. 🌐 **public**

- **Meaning:** Open for all⁷⁵.
- **Accessibility:** A public member can be accessed from anywhere in the program—from within the same class, from other classes in the same package, and from classes in different packages⁷⁶⁷⁶⁷⁶⁷⁶.

- **Example:**

```
Java  
public class Student {  
    public String name; // Accessible from any other class [cite: 234]  
    public void showName() { /* ... */ } [cite: 235]  
}
```

2. 🔒 **private**

- **Meaning:** Very restricted⁷⁷.

- **Accessibility:** A private member can only be accessed from **inside the same class** where it is declared⁷⁸. It is not visible to any other class, not even its subclasses⁷⁹⁷⁹⁷⁹⁷⁹.

- **Example:**

Java

```
class Student {
    private int age; // Can only be used inside the Student class [cite: 225]
    private void showAge() { /* ... */ } [cite: 226]
}
```

3. default (No Keyword)

- **Meaning:** Package-level access⁸⁰.
- **Accessibility:** If you don't use any modifier, it is treated as default. A default member can be accessed only by classes **within the same package**⁸¹⁸¹⁸¹⁸¹. It cannot be accessed from outside the package⁸².

- **Example:**

Java

```
class Student {
    String city; // default access: visible to classes in the same package [cite: 246]
    void showCity() { /* ... */ } [cite: 247]
}
```

4. protected

- **Meaning:** A special case for inheritance⁸³.
- **Accessibility:** A protected member is accessible **within its own package** (like default) and also by **subclasses (child classes) in other packages**⁸⁴⁸⁴⁸⁴⁸⁴.

- **Example:**

Java

```
class Student {
```

```
protected String grade; [cite: 260]
}

// In another package
class Child extends Student {
    void showGrade() {
        System.out.println(grade); // Can access 'grade' because Child is a subclass
    }
}
```

21. Explain abstract classes and methods.

Abstraction is the concept of hiding implementation details and showing only functionality⁸⁵. In Java, abstraction is primarily achieved using abstract classes and interfaces.

Abstract Method

An

abstract method is a method that is declared but has no implementation—it has no body⁸⁶. It is created using the

abstract keyword and ends with a semicolon instead of curly braces {}⁸⁷.

- **Key Points:**

- It only has a method signature, no code inside.
- It **must** be declared inside an abstract class⁸⁸.

- Any class that inherits from the abstract class must provide an implementation (i.e., override) for all its abstract methods.

- **Example:**

Java

```
abstract void draw(); // This is an abstract method [cite: 1023]
```

Abstract Class

An

abstract class is a class that is declared with the `abstract` keyword⁸⁹⁸⁹⁸⁹⁸⁹. It serves as a template for other classes.

- **Key Points:**

- It **cannot be instantiated**, meaning you cannot create an object of an abstract class directly⁹⁰.
- It can contain both **abstract methods** (methods without a body) and **regular methods** (concrete methods with a body)⁹¹⁹¹⁹¹⁹¹.
- It is designed to be subclassed (inherited from).

Example

Here is an example of a Shape abstract class with both an abstract method and a regular method.

Java

```
// 1. Abstract Class
```

```
abstract class Shape { [cite: 1022]
```

```
    // Abstract method - must be implemented by subclasses
```

```
    abstract void draw(); [cite: 1023]
```



```
// Regular method - subclasses can use this as-is
void show() { [cite: 1024]
    System.out.println("This is a shape"); [cite: 1025]
}
}
```

// 2. Concrete Subclass

```
class Rectangle extends Shape { [cite: 1029]
    // Providing the implementation for the abstract draw() method
    void draw() { [cite: 1030]
        System.out.println("Drawing Rectangle");
    }
}
```

// 3. Main Class

```
public class Test {
    public static void main(String[] args) { [cite: 1032]
        // We can't do "new Shape()", but we can have a Shape reference
        // to a concrete subclass object.
        Shape s = new Rectangle(); [cite: 1033]

        s.draw(); // Calls the overridden method in Rectangle. Output: Drawing Rectangle [cite: 1034]
        s.show(); // Calls the regular method from Shape. Output: This is a shape [cite: 1035]
    }
}
```